

Bussiness Galore, technical report

Catalina Ariza Ardila — Julian David Celis Giraldo
Department of System Enginner
Francisco José de Caldas District University

CONTENTS

I	Introduction	1
II	User stories	1
II-A	Manager (Administrator)	1
II-B	Customer (User)	1
III	Requirements	2
III-A	Functional Requirements	2
III-B	non -Functional Requirements	2
IV	Technical decisions	3
V	Object-oriented principle analysis	3
VI	Analysis and Details of the Implementation of Design Patterns	3
VII	Conclusions	4
	References	4

LIST OF FIGURES

1	Class Diagram	3
2	CRC Cards	3

LIST OF TABLES

Bussiness Galore, technical report

Abstract—AI final

I. INTRODUCTION

In modern application development, microservice-based architecture allows for improved scalability and maintainability of systems. In this context, this report describes the design and implementation of reservation management microservices, developed with FastApi, which communicates with other services written in Java by exchanging data in JSON format.

One of the main challenges in this system is to guarantee data integrity and avoid data conflicts. To address this problem, different mechanisms based on design patterns have been implemented, allowing business rules to be modularized and organized efficiently.

This report details the structure of the microservice, the validation logic applied to reservations, and the integration with other system components. In addition, the tools used are presented, such as Poetry for dependency management and Pylint to ensure code quality.

II. USER STORIES

User stories were defined to define user requirements, based on the roles, we analyzed criteria to approve the user stories.

A. Manager (Administrator)

1. **As a manager, I want** to register my shop on the application **so that** customers can learn about my business.

Acceptance Criteria

- The administrator must be able to register their shop.
- The user must be able to enter shop information (location, type of business, service hours).
- Customers must be able to view this information.
- The system must validate if the administrator already owns a shop.

2. **As a manager, I want** to have a space to list services, details, and the service hours, so that customers are well informed about these aspects.

Acceptance Criteria

- The administrator must be able to name the services.
 - The administrator must be able to assign service details and prices.
 - The services should appear in the services list.
 - Customers must be able to select the time for their service.
3. **As a manager, I want** to have a space to showcase in detail the works or services offered in my shop, so that customers can check if it's what they are looking for.

Acceptance Criteria

- The administrator must be able to edit, add, or delete photos of the completed works.
 - Photos must appear in an accessible and visible section for customers.
4. **As a manager, I want** to be able to show basic information about my employees on the shop's profile, so that customers can see relevant details and choose who they want to schedule with.

Acceptance Criteria

- The administrator must be able to add employees to the shop's profile, including basic information (name, photo, specialization).
 - Customers must be able to see basic employee information and select a professional according to their preference.
 - The employee's personal information must be visible both on the services page and the shop's profile.
5. **As a manager, I want** to register to have my profile in the app and add basic personal information.

Acceptance Criteria

- The user must be able to fill out their data (name, birthdate, gender, phone number).
- The basic information must appear on the user profile and be editable at any time.
- The data must be displayed on the user's profile.
- The user must receive a confirmation that their profile has been successfully created.
- If any required fields are missing, the system must display a clear error message.

B. Customer (User)

1. **As a customer, I want** to search for shops by name or code for convenience.

Acceptance Criteria

- The customer must be able to easily find and view the shops the application offers
- in relation to what they are searching for.
- The information should appear summarized so that the customer can quickly find what they want.
- The customer must be able to search for a shop by name or code

- The system must show a message when no shops match the customer's search terms.
2. **As** a customer, **I want** to schedule one or more services to be attended.

Acceptance Criteria

- The customer must be able to view the availability of time slots before booking a service.
 - The system must validate that the selected time slot is not occupied by another customer.
 - The customer must receive confirmation of their appointment once the service booking is completed.
3. **As** a customer, **I want** to see a summary of my appointments, both past and future, to keep track of the services I've taken and the ones I have scheduled.

Acceptance Criteria

- The customer must be able to see their upcoming appointments, with the date, time, location, and the booked service.
- The customer must be able to see a history of completed appointments, with information about the date and the service rendered.
- The customer must be able to access these sections from their profile or an appointments menu.

III. REQUIREMENTS

There are two types requirements, functional and non-functional requirements

A. Functional Requirements

We have organized the functional requirements according to the user and their interaction with the system as follows:

1. Shop management

- The administrator should be able to register their shop in the app.
- The administrator should be able to enter the shop information (location, business type, hours of operation).
- The system should validate whether the administrator already has a registered shop before allowing creation.
- Customers should be able to view shop information in the app.

2. Service Management

- The administrator should be able to add, edit, and delete services offered by their shop.
- Services should include name, details, and prices.
- Customers should be able to view the list of available services.
- Customers should be able to select an available time for their service.

4. Employee Management

- The administrator should be able to add employees to their shop with basic information
- Customers should be able to view employees and select who they want to schedule with.
- Employee information should be displayed on both the services page and the store profile.

5. User Registration

- The user should be able to register by entering their name, date of birth, gender and phone number.
- The system should display the user's information on their profile and allow editing.
- The user should receive a confirmation after completing their registration.
- If any required field is missing, the system should display an error message.

6. Store Search

- Customers should be able to search for stores by name or code.
- The system should display a list of relevant results sorted by proximity or match with the search.
- If no stores are found, the system should display a message indicating that there are no matches.

7. Service Reservation

- Customers should be able to view the availability of times before making a reservation.
- The system should validate that the selected time slot is not occupied.
- The customer should receive a confirmation of the appointment upon completing the reservation.

8. Appointment History and Tracking

- Customers should be able to view their upcoming appointments with date, time, and booked service.
- Customers should be able to view past appointment history with details of the service received.

B. non -Functional Requirements

Performance: Store and service searches must respond in under 2 seconds under normal load conditions.

Access Control: Only authorized administrators should be able to modify store, employee, and service information.

Authentication and Authorization: The system must include a role-based authentication and authorization mechanism.

Code Quality: The code must adhere to development standards and best practices to ensure easy maintenance.

Scalability and Extensibility: The system should support integration with future modules or services without affecting existing functionalities.

Documentation: Clear and up-to-date developer documentation must be maintained.

IV. TECHNICAL DECISIONS

Performance and Scalability

Store and service searches must respond in less than 500 ms.

The application must be scalable to handle growth in the number of stores and customers without loss of performance.

Maintenance and Extensibility

The system should be modularized to facilitate maintenance and add new functionality without affecting existing functionality.

3. Integration

The application should communicate with other systems using REST APIs and JSON.

The microservice should support FastAPI (Python) and communicate with other microservices in Java using JSON

The API should follow RESTful standards to facilitate integration with other systems.

The microservice should be horizontally scalable to support increased load if necessary. In case of failure, the system must automatically recover in less than 1 minute.

The system must be available 99.9%. Other aspects we will take into account:

As a team, we decided to limit the aggregation of multiple shops to a single administrator, as this would add a level of complexity to the application.

We also decided that the "Manager" class should be independent of the "Employee" class due to the amount of responsibilities the manager has.

Since we don't have clients yet, we tried to create the user stories based on our own experiences.

As a team we decided that the employee belongs only to one shop.

V. OBJECT-ORIENTED PRINCIPLE ANALYSIS

The object-oriented paradigm allows us to represent real-world ideas in software as realistically as possible, using abstraction and concepts of class-object. In this project, this paradigm is very useful because the application aims to solve a real-world problem (scheduling appointments in different shops in a fast and organized way). Additionally, since this is a complex and scalable system, OOP's modular approach to organizing and structuring the code makes problem development easier. In our project, several OOP concepts are already implemented, such as polymorphism when scheduling a service, as the process varies depending on the shop. The SOLID principles are also followed, which can be seen in the application's class diagram.

S: The classes comply with the single responsibility principle. This is best observed in the CRC cards that were created.

O: By implementing inheritance, the abstract class meets the criterion of being closed to modification, and the concrete classes comply with the criterion of being open to extension.

L: By implementing inheritance, the subclasses can be used as if they were the superclass, thus fulfilling Liskov's substitution principle.

I: The principle of interface segregation is not being violated, although it is not implemented either because there are no interfaces.

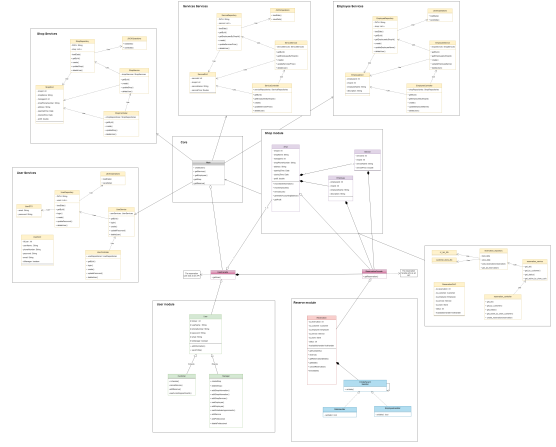


Fig. 1. Class Diagram

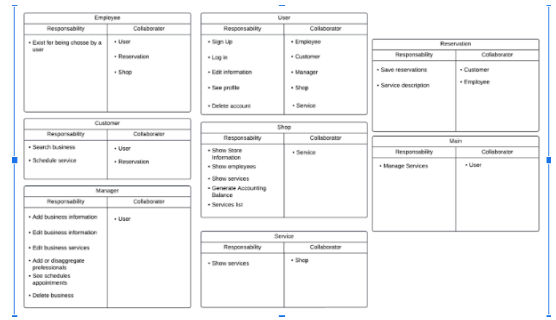


Fig. 2. CRC Cards

D: The user only connects to high-level classes, thus ensuring the dependency inversion principle.

As the project progresses, we aim to implement design patterns, which are an important part of programming with OOP.

VI. ANALYSIS AND DETAILS OF THE IMPLEMENTATION OF DESIGN PATTERNS

We initially decided to implement two design patterns: the Chain of Responsibility (a creational pattern) and the Facade (a structural pattern). Chain of Responsibility Pattern: This was implemented because our program required multiple validations for users to schedule appointments, where each validation depended on the previous one. This pattern is perfect for handling such a process. Facade Pattern: Although implemented as an API in the project, its functionality remains the same. It enables communication between different application modules, keeping complex logic hidden while reducing multiple dependencies between them. Why Didn't We Use the Others? Creational Patterns (Builder, Factory Method, Abstract Factory, Singleton, Prototype): At this stage, our program does not require constructing large quantities of different objects, nor does it demand complex object creation. Implementing a creational pattern would add unnecessary complexity to our project.

Structural Patterns (Bridge, Composite, Flyweight, Decorator, Adapter):

- Some of these patterns have very specific applications that do not fit our project. Others, such as Decorator and Adapter, do not currently contribute significantly to the first version of the program.
- Proxy: This pattern could be beneficial in our current design, as it helps manage logs and histories more efficiently, making the code more flexible and reducing memory usage. However, we decided to omit its implementation in this first release due to the limited time available to ensure high-quality integration.

Behavioral Patterns

- Iterator: We decided not to use this pattern since we do not need to iterate through lists in a special manner.
- Memento: This pattern could allow users to revert to previous versions while scheduling an appointment to make quick corrections. However, it is not essential for this first release.
- Strategy, Template, State, Mediator, Command, Observer: These patterns could become useful as the program grows. However, implementing them prematurely would introduce unnecessary complexity, falling into the realm of antipatterns.

Analysis of Best Programming Practices and Antipatterns

We made a strong effort to adhere to best programming practices and avoid antipatterns. As discussed in the previous section, we carefully analyzed which design patterns were truly necessary for the first version of the program to prevent overengineering.

Regarding Code Smells, while we cannot guarantee the complete absence of them, we took proactive steps to identify and correct issues as they arose during development. Some of the code smells we addressed include:

- Spaghetti code
- Hardcoded values and magic numbers
- Large classes and methods
- Refused request
- Speculative generality

VII. CONCLUSIONS

Throughout the development of this document, the importance of design patterns, software architecture, and best programming practices has been evident. When properly implemented, these elements contribute to a more maintainable, robust, flexible, and clean codebase, among other benefits.

The use of UML diagrams, user stories with acceptance criteria, and CRC cards has streamlined code construction, making the development process faster and more structured. These tools have helped ensure that the final product aligns closely with the needs and expectations of the end user.

By leveraging APIs, we were able to develop our project across different programming environments, allowing us to

work more efficiently while taking advantage of the best features each environment has to offer.

One of the most critical aspects of application development is organization, especially in team-based projects. Implementing an agile methodology significantly improved our workflow and collaboration throughout the project.

Applying SOLID principles in object-oriented projects is fundamental to achieving a well-structured and scalable program.

Additionally, the use of Docker simplified the creation, deployment, and management of our application through containerization. This was particularly useful in our case, where multiple backends needed to interact seamlessly. Docker's scalability also makes it an ideal choice for microservices-based applications like ours.

Other known code smells were avoided from the start. Additionally, we adhered to the Law of Demeter and the Principle of Least Knowledge, which help reduce coupling between classes, making the code more maintainable and modular.

REFERENCES

- [1] Beunik. "Beunik: Reserve in Beauty Salons and Barbershops." [Online]. Available: <https://beunik.co/>. [Accessed: Oct. 3, 2024].
- [2] C. A. Martínez, "Características de la Programación Orientada a Objetos," *Cibernética 1*. [Online]. Available: <https://portalacademico.cch.unam.mx/cibernetica1/algoritmos-y-codificacion/caracteristicas-POO> [Accessed: Oct. 3, 2024].